2 Regression

Regression is a fundamental tool in machine learning used to model the relationship between an independent variable (called data and denoted x) and a dependent variable (called target and denoted y). The goal is to learn a function that best predicts the target value from the input data. This relationship is shown in figure 2.1 .

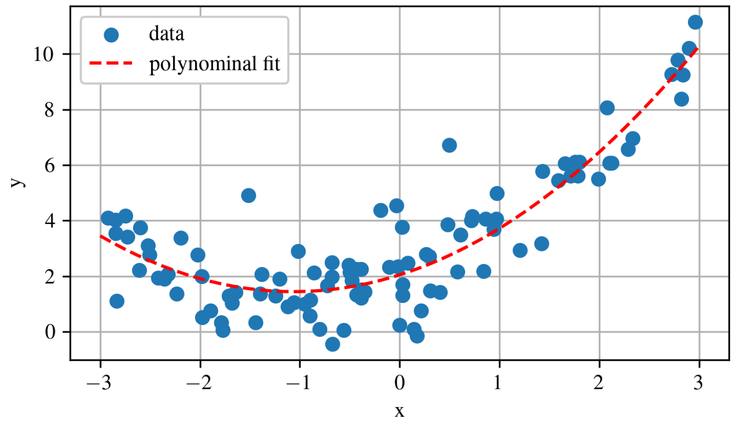

Figure 2.1: Polynomial model fit via regression to a generic dataset.

In this chapter, we first examine Linear Regression and contrast two ways to estimate its parameters:

1. **Closed-form solution.** Solve for the parameter vector in one step by minimizing the cost function on the entire training set.

2. **Gradient Descent .** Apply an iterative optimizer that repeatedly updates the parameters in small steps that lower the cost until it reaches the same optimum found by the closed-form method. We will look at three common Gradient Descent variants: Batch Gradient Descent, Mini-batch Gradient Descent, and Stochastic Gradient Descent.

The first approach gives an exact answer immediately, whereas the second reaches that answer through successive refinements.

Review 2.1 Ames Housing Dataset
The Ames housing dataset provides details on individual residential property sales in Ames, Iowa, from 2006 to 2010 [a]. It includes 2,930 entries and numerous variables (23 nominal, 23 ordinal, 14 discrete, and 20 continuous) that are used to evaluate home values [b].

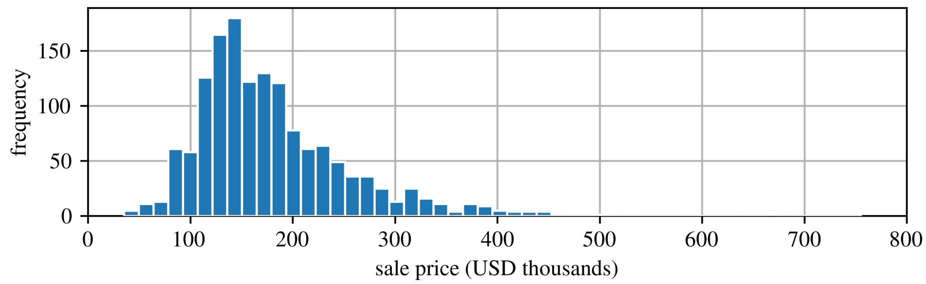

Figure 2.2: Histogram of sale prices from 2006 to 2010 for the 2,930 entries.

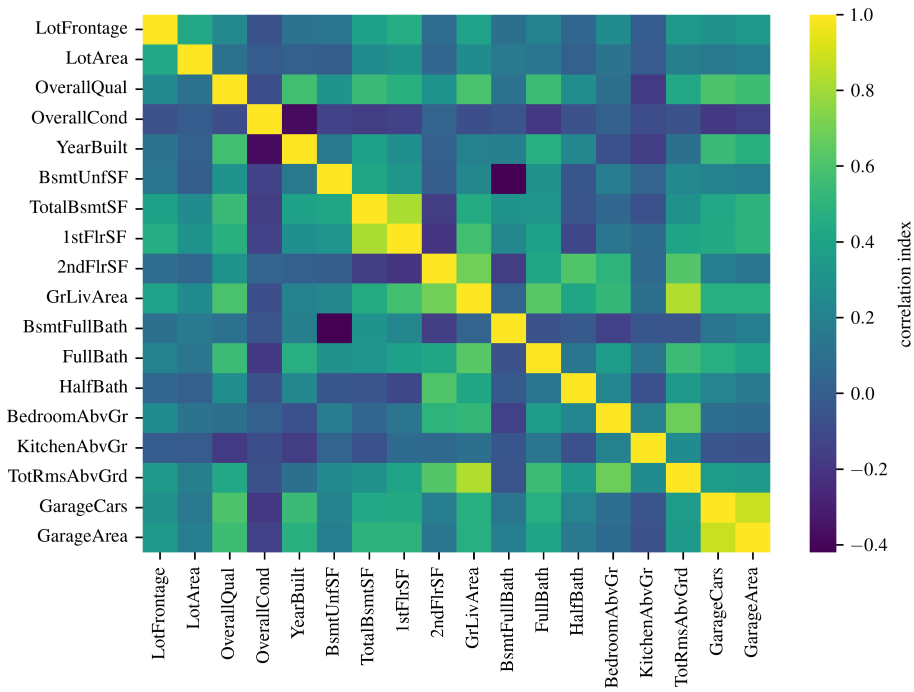

Figure 2.3: Correlation matrix for the 13 input parameters.

[a] The author of this text bought a home in Ames Iowa during this time right as the 2008 housing crash unfolded and on the last day they allowed home loans with no down payment.

[b] De Cock, Dean. "Ames, Iowa: Alternative to the Boston housing data as an end of semester regression project." Journal of Statistics Education 19.3 (2011).

2.1 Linear Regression

Consider the scenario where you wish to determine the value of a house in Ames using the Ames dataset included in sklearn. To illustrate this, let us plot the relationship between the above ground living area and the housing price.

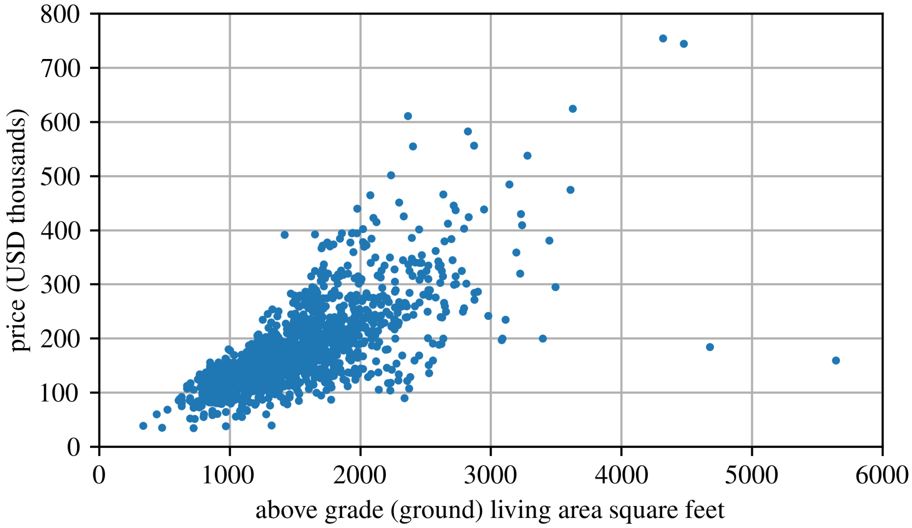

Figure 2.4: Ames housing data.

A clear trend is observable, despite the data's noise (i.e., partial randomness): the value of a house increases with the above ground living area. Therefore, the house value can be modeled as a linear function of the above ground living area:

$$\text{price\_model} = \theta_1 + \theta_2 \times \text{above\_ground\_living\_area} \tag{2.1}$$

This model comprises two parameters, θ_1 and θ_2, which can be adjusted to represent any linear relationship.

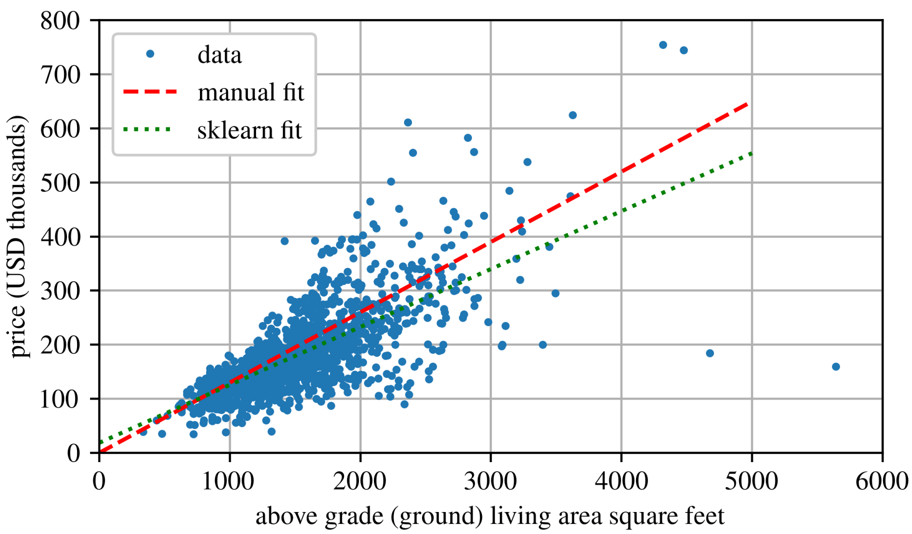

Figure 2.5: Ames housing data with trend lines.

Notations

In terms of linear algebra notation; we use:

- $\mathbf{x}$: lowercase bold symbols denote vectors.
- $\mathbf{X}$: uppercase bold symbols denote matrices.

In machine learning, we commonly use:

- $\mathbf{X}$: the feature matrix (input data).
- $\mathbf{y}$: the target vector (labels or outputs).
- $\mathbf{x}^{(i)}$: the i^{th} instance in the dataset.
- h: the hypothesis or prediction function, $\hat{\mathbf{y}} = h(\mathbf{X})$.
- n: the number of features.
- m: the number of training instances.
- $\hat{x}$: an estimated value ("x-hat").

2.1.1 Closed Form Solution

Before you can tune a model's parameters, you need a clear way to judge how well it fits the data. The standard approach is to define a cost function that assigns a numerical penalty to poor fits; lower values indicate better performance. For a linear model, this cost function measures the difference between the model's predictions and the actual training targets, and you minimize that difference using Ordinary Least Squares linear regression. Practically, you feed your training data into the algorithm, which then solves for the parameter values that align the linear relationship most closely to the data. In Python, this procedure is implemented in `scikit-learn` as `sklearn.linear_model.LinearRegression`.

In the context of the Ames housing data, a straightforward cost function used is the Mean Squared Error (MSE), defined as

$$\text{MSE} = J = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 \tag{2.2}$$

where m is the number of dataset instances, and J serves as a general representation of the cost function in machine learning contexts.

The MSE is also applied to minimize the error in a linear regression model, with the hypothesis $h_{\boldsymbol{\theta}}$, trained on dataset $\mathbf{X}$. This is expressed as:

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = J = \frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2 \tag{2.3}$$

where the objective is to minimize the cost function by iteratively refining the values of $\boldsymbol{\theta}$.

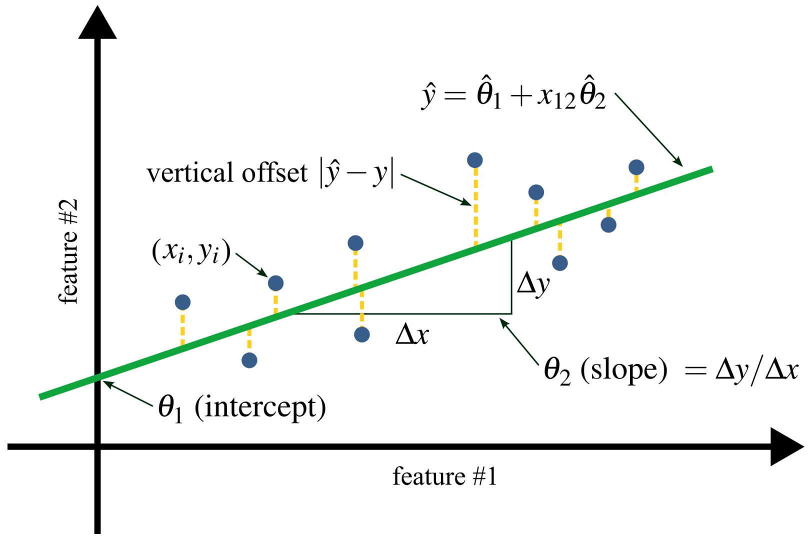

Figure 2.6: Linear regression of a two-feature dataset obtained with least squares.

Consider a dataset containing n observations, denoted as $(y_i, x_i)_{i=1}^n$. Each observation i consists of a scalar response y_i and a column vector x_i that holds the values for p predictors (regressors), represented as x_{ij} where $j = 1, \ldots, p$. In the context of a linear regression model, the response variable y_i is modeled as a linear combination of the regressors, influenced by an error term ε_i:

$$y_i = \theta_1 x_{i1} + \theta_2 x_{i2} + \ldots + \theta_y x_{im} + \varepsilon_i \tag{2.4}$$

This model can also be written in matrix notation as

$$\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \varepsilon \tag{2.5}$$

Where:

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \ldots & x_{1m} \\ x_{21} & x_{22} & \ldots & x_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \ldots & x_{nm} \end{bmatrix}, \boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix}, \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \tag{2.6}$$

However, the best-fit model $\hat{y}$ will not be able to account for the unmodeled error, therefore:

$$\hat{y}_i = \hat{\theta}_1 x_{i1} + \hat{\theta}_2 x_{i2} + \ldots + \hat{\theta}_m x_{im} \tag{2.7}$$

or

$$\hat{\mathbf{y}} = \mathbf{X}\hat{\boldsymbol{\theta}} \tag{2.8}$$

This goal is to find $\hat{\theta}$ such that the error is minimized. Mathmatically, this is simple enough as the $\hat{\theta}$ Is the minimization of the lease-squares hyperplane, or:

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \tag{2.9}$$

Equation 2.9 is referred to as the normal equation. Consider a simple linear model with $p = 2$, where $x_{11} = 1$. In this scenario, x_{11} acts as the bias term of the equation, whereas the remaining elements of $\mathbf{X}$ represent the data. Without this bias term, the solution would only address the slope of the line. Subsequently, solving for $\hat{\theta}_1$ and $\hat{\theta}_2$ yields:

$$[x_{11}x_{12}][\hat{\theta}_1\hat{\theta}_2] = \hat{y} \tag{2.10}$$

As $x_{11} = 1$ (again, called the bias term) this becomes:

$$\hat{y} = \hat{\theta}_1 + x_{12}\hat{\theta}_2 \tag{2.11}$$

This is simply another expression for the equation of a liner line:

$$y = mx + b \tag{2.12}$$

Solving the model output for a input or a series of inputs can than be done simply enough.

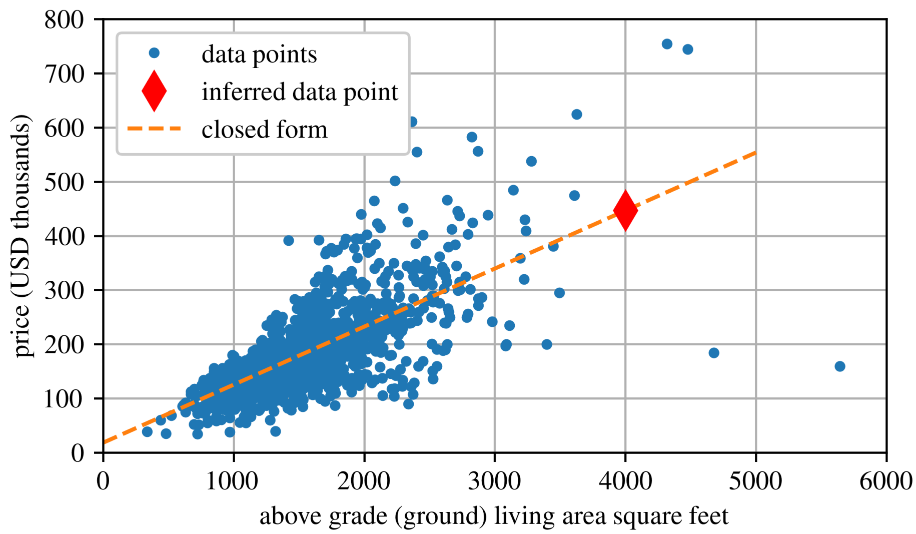

Figure 2.7: Ames housing data inferred.

2.1.2 Computational Complexity

The Normal Equation calculates the inverse of $\mathbf{X}^\top\mathbf{X}$, an $n \times n$ matrix, where n represents the number of features. The computational complexity of matrix inversion generally ranges from $O(n^{2.4})$ to $O(n^3)$, depending on the specific algorithm used. Consequently, doubling the number of features would increase the computational effort by approximately $2^{2.4} \approx 5.3$ to $2^3 = 8$ times.

WARNING

When the number of features becomes very large, for example around $100,000$, solving the Normal Equation becomes extremely slow.

On the positive side, the computational complexity of this equation is linear with respect to the number of instances in the training set, denoted as $O(m)$. This allows it to efficiently handle large training sets, as long as they fit within memory.

Furthermore, once your Linear Regression model is trained (whether through the Normal Equation or another method), making predictions is very quick. The computational effort is linear in relation to both the number of instances you wish to predict and the number of features. Essentially, predicting for twice as many instances or features will approximately double the computation time.

Next, we will explore different methods for training a Linear Regression model that are more appropriate for scenarios with a large number of features, or when the training data exceeds memory capacity.

Example 2.1 Linear Regression - Ames Housing Dataset
This example uses the Ames housing dataset to model the relationship between above-ground living area and house price using linear regression. It compares a manual fit, a closed-form solution, and gradient descent, alongside Scikit-Learn's `LinearRegression` and `SGDRegressor`, to illustrate how each method fits a line to the data.

2.2 Gradient Descent

Gradient Descent is a versatile optimization algorithm capable of finding optimal solutions across a broad spectrum of problems. The core concept of Gradient Descent involves iteratively adjusting parameters to minimize a cost function [a]. Consider the analogy of being lost in a dense fog in the mountains and seeking to descend:

1. You sense the ground's slope under your feet (the local gradient).
2. You descend in the direction of the steepest descent.
3. Reaching a point where the local gradient is zero indicates that you have found a minimum.

The Gradient Descent process starts with random initialization of the parameter θ, followed by gradual improvements. Each incremental step aims to reduce the cost function until the algorithm converges to a minimum.

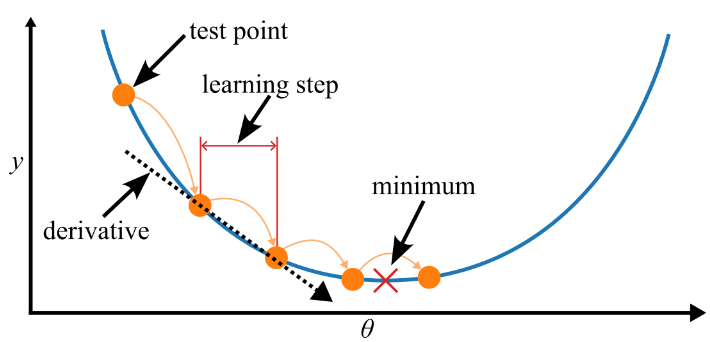

Figure 2.8: Illustration of gradient descent.

A critical aspect of Gradient Descent is the step size, controlled by the learning rate hyperparameter. A small learning rate leads to a slow convergence, requiring many iterations. Conversely,

[a] Ruder, Sebastian. "An overview of gradient descent optimization algorithms." arXiv preprint arXiv:1609.04747 (2016).

a high learning rate might cause the algorithm to overshoot the minimum, potentially causing divergence and failing to find an optimal solution.

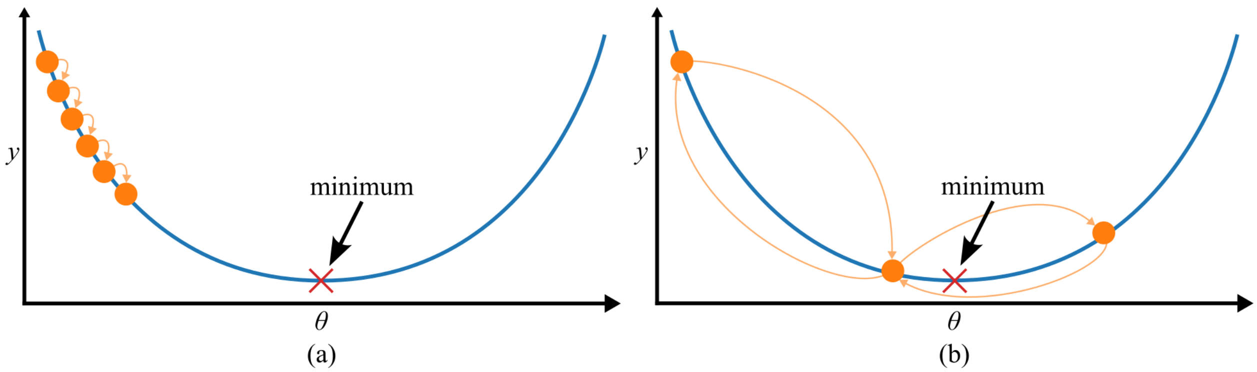

Figure 2.9: Effect of learning rate in gradient descent, showing: (a) a slow search with a low step size, and (b) a search with a large step size that oscillates around the minimum without ever finding the minimum.

Cost functions do not always present themselves as neat, concave shapes. They can feature obstacles such as holes, ridges, plateaus, and various complex topographies that complicate the convergence to the minimum. The challenges associated with Gradient Descent include:

- Starting the algorithm from a random point on the left may lead to convergence at a local minimum rather than the more optimal global minimum.
- Initiating on the right might result in a prolonged journey across a plateau, and terminating the process too soon could prevent reaching the global minimum.

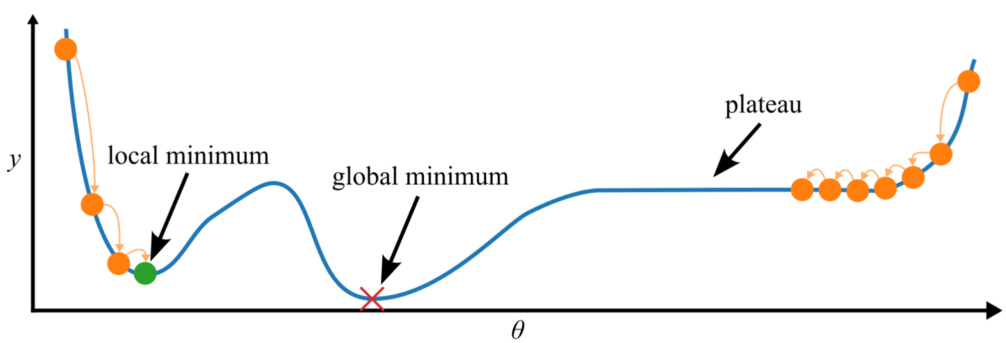

Figure 2.10: Gradient features and descent challenges.

Fortunately, the Mean Squared Error (MSE) cost function for Linear Regression models is convex. This characteristic ensures that for any two points on the curve, the line segment joining them does not intersect the curve itself. Consequently:

- There are no local minima, only one global minimum exists.
- It is a continuous function, and its slope changes smoothly.

These properties significantly benefit the optimization process: Gradient Descent can reliably approximate the global minimum given sufficient time and an appropriate learning rate.

2.2.1 Comparison of Gradient Descent Methods

Before looking into the mathematical details, let us examine how various gradient descent methods achieve the optimal solution, as illustrated in Figure 2.11. Additionally, Table 1 provides a comparative analysis of different gradient descent methods, highlighting their respective strengths and weaknesses.

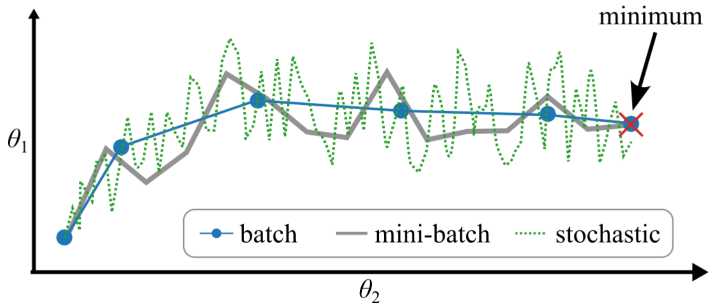

Figure 2.11: Comparing gradient descent methods.

Table 1: Comparison of various linear regression solutions.

algorithm	large number of instances (m)	data must fit in memory	large number of features (n)	hyperparams	scaling required	Scikit-Learn
normal equation	fast	yes	slow	0	no	`LinearRegression`
batch GD	slow	yes	fast	2	yes	n/a
stochastic GD	fast	no	fast	≥ 2	yes	`SGDRegressor`
mini-batch GD	fast	no	fast	≥ 2	yes	n/a

NOTE

After training, there is minimal difference between the models: these algorithms converge on similar solutions and make predictions in a nearly identical manner.

2.2.2 Batch Gradient Descent

Batch Gradient Descent (shown in figure 2.12) processes the entire batch of training data $\mathbf{X}$ at every step, which is the origin of its name, “batch.” Consider again our Mean Squared Error (MSE) cost function:

$$J = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2 \tag{2.13}$$

For a hypothesis h_θ, the MSE for the dataset $\mathbf{X}$ can be computed for each instance $\mathbf{x}^{(i)}$ as follows:

$$J(\mathbf{X}, h_\theta) = J = \frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)})^2 \tag{2.14}$$

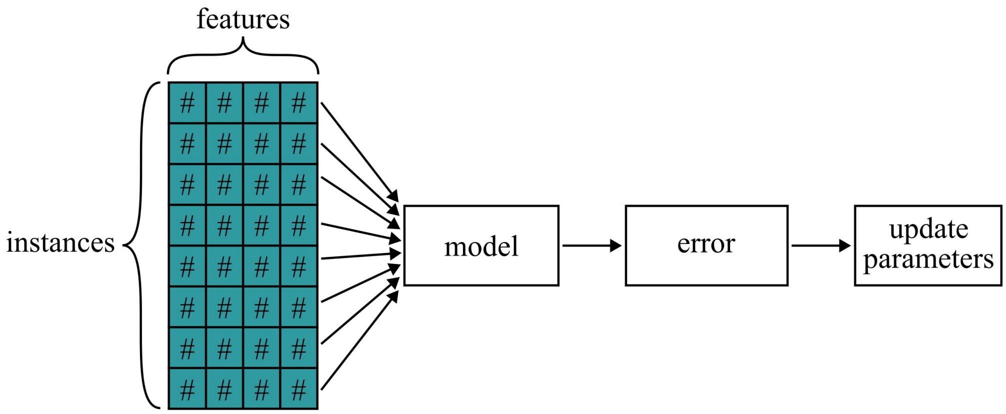

Figure 2.12: Flowchart of Batch Gradient Descent.

With this foundation, we can implement gradient descent by computing the gradient of the cost function with respect to each parameter θ_j. Specifically, this involves calculating how much the cost function changes when θ_j is altered slightly. This calculation is known as a partial derivative. Conceptually, it is akin to determining "the slope of the mountain under my feet if I face east," then repeating the question facing north, and similarly for any other direction in a higher-dimensional space. The partial derivative of the cost function with respect to parameter θ_j is computed as:

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{2}{m} \sum_{i=1}^{m} (\boldsymbol{\theta}^\top \mathbf{x}^{(i)} - y^{(i)}) x_j^{(i)} \tag{2.15}$$

Rather than computing each partial derivative individually, all partial derivatives can be calculated simultaneously using the gradient vector, $\nabla_\theta J(\boldsymbol{\theta})$, which encompasses all partial derivatives of the cost function for each model parameter:

$$\nabla_\theta J(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} J(\theta) \\ \frac{\partial}{\partial \theta_1} J(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} J(\theta) \end{pmatrix} = \frac{2}{m} \mathbf{X}^\top \cdot (\mathbf{X}\boldsymbol{\theta} - \mathbf{y}) \tag{2.16}$$

WARNING

Be aware that this method computes using the entire training set **X** at every step of Gradient Descent, which is why the approach is named Batch Gradient Descent. Although this method can be exceedingly slow with large training sets, it performs well with numerous features, outpacing the Normal Equation in training Linear Regression models with extensive features.

Once you obtain the gradient vector, which indicates the direction of ascent, the next step involves moving in the opposite direction, effectively going downhill. This is achieved by subtracting $\nabla_{\theta} J(\theta)$ from $\boldsymbol{\theta}$, incorporating the learning rate η to scale the step size:

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\theta} J(\boldsymbol{\theta}) \tag{2.17}$$

Interestingly, this process aligns perfectly with the results from the Normal Equation. However, variations in the learning rate η can significantly influence the outcomes. Figure 2.13 illustrates the first 10 steps of Gradient Descent with three different learning rates, with the dashed line marking the starting point.

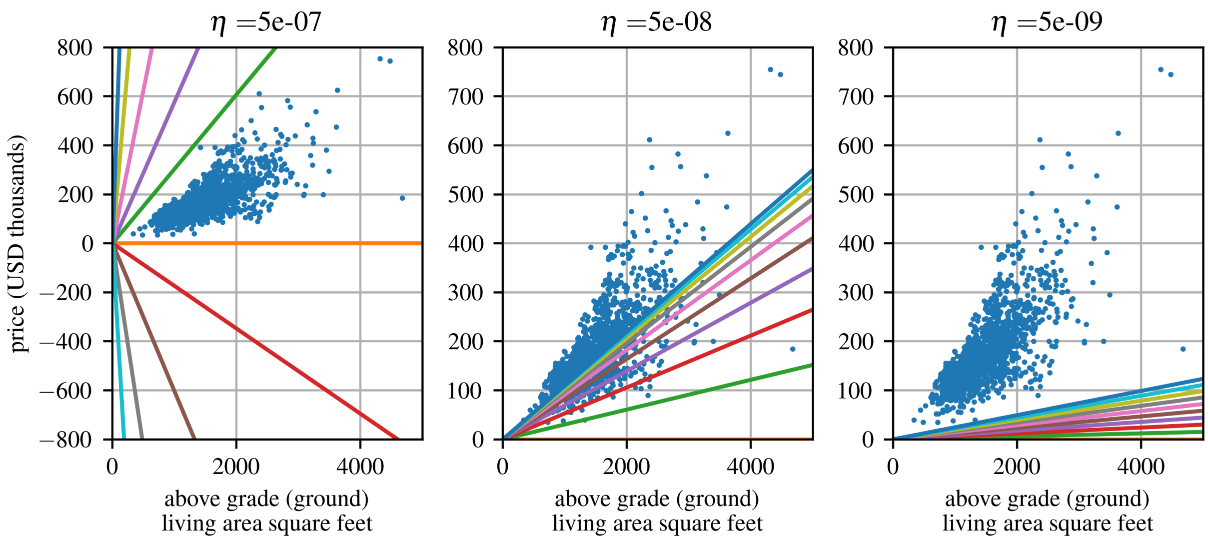

Figure 2.13: The effects of different learning rates on the gradient descent algorithms, showing: (a) too high ($\eta = 5e-07$); (b) about right ($\eta = 5e-08$), and; (c) too low ($\eta = 5e-09$).

Figure 2.13 highlights the impact of the learning rate on gradient descent. Figure 2.13(a) With a very large step size, $\eta = 5 \times 10^{-7}$, each update overshoots the minimum so the algorithm diverges rather than converging. Figure 2.13(b) Using $\eta = 5 \times 10^{-8}$ provides a balanced step size; the cost decreases just right [a] and the algorithm reaches the optimum in only a few iterations. Figure 2.13(c) A small step size, $\eta = 5 \times 10^{-9}$, keeps the algorithm moving toward the minimum yet progress is slow, requiring many more iterations to achieve the same result.

To identify an appropriate learning rate, a grid search can be employed. It is advisable to limit the number of iterations during grid search to avoid models that are too slow to converge.

Determining the correct number of iterations is also crucial. If set too low, the algorithm may stop far from the optimal solution. Conversely, overly high settings lead to unnecessary computations after convergence. A practical approach is to allow a large number of iterations but to halt the algorithm when the gradient vector's norm shrinks to below a small threshold ε (known as the tolerance), indicating proximity to the minimum.

[a] Pyle, K. "Goldilocks and the three bears." Mother's Nursery Tales (1918): 207-213.

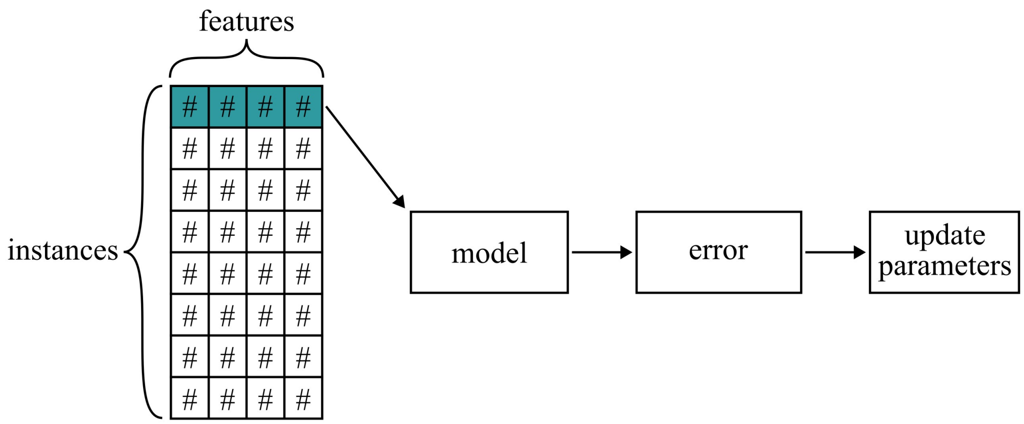

Figure 2.14: Flowchart of Stochastic Gradient Descent.

2.2.3 Stochastic Gradient Descent

Stochastic Gradient Descent (shown in figure 2.14) updates the model parameters after evaluating each individual training sample $x^{(i)}$ and its corresponding label $y^{(i)}$, which stands in contrast to batch gradient descent that uses the entire dataset for each update. The update rule for SGD can be expressed as

$$\theta^{(\text{next step})} = \theta - \eta \nabla_\theta J(\theta; x^{(i)}; y^{(i)}). \tag{2.18}$$

While batch gradient descent tends to perform unnecessary recalculations for large datasets by reevaluating gradients for similar examples with each update, SGD eliminates this inefficiency by updating parameters incrementally. Consequently, SGD is generally faster and adaptable for online learning. Due to its frequent and individual updates, SGD exhibits high variance in the objective function, causing significant fluctuations as illustrated in Fig. 2.15(b).

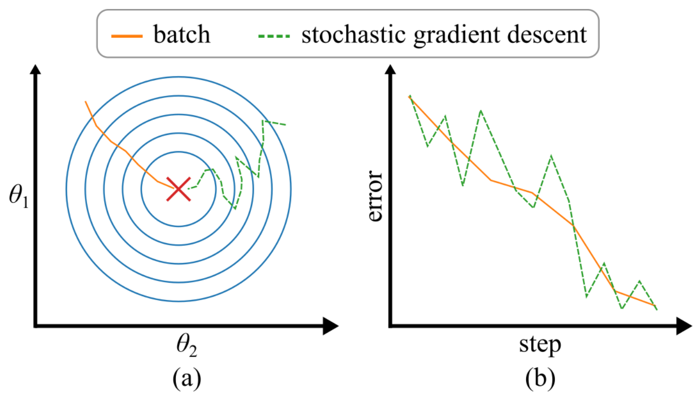

Figure 2.15: Comparison of the iterative performance of Batch and Stochastic Gradient Descent, showing: (a) how they move towards their target, and (b) the error at comparable steps.

2.2.4 Mini-batch Gradient Descent

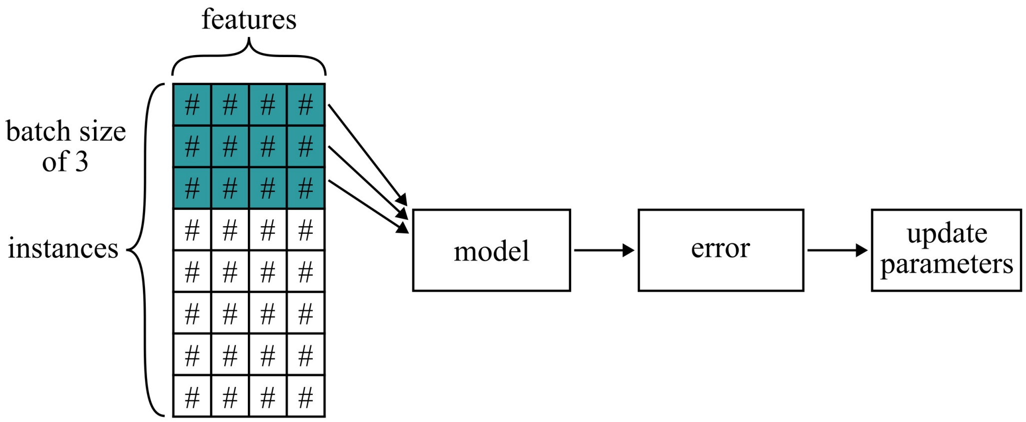

Figure 2.16: Flowchart of Mini-batch Gradient Descent.

$$\theta^{(\text{next step})} = \theta - \eta \nabla_\theta J(\theta; x^{(i:i+n)}; y^{(i:i+n)}) \tag{2.19}$$

The final Gradient Descent algorithm we will discuss is Mini-batch Gradient Descent. This method combines elements of both Batch and Stochastic Gradient Descent: rather than calculating gradients using the entire training set (as in Batch GD) or a single instance (as in Stochastic GD), Mini-batch GD computes gradients using small, randomly selected subsets of instances known as minibatches. One significant benefit of Mini-batch GD over Stochastic GD is the improved performance from hardware-optimized matrix operations, particularly on GPUs.

Mini-batch GD's trajectory through parameter space tends to be more stable compared to the often erratic path of SGD, especially with larger minibatches. This stability typically brings Mini-batch GD closer to the minimum than SGD. However, Mini-batch GD might struggle more with escaping local minima in scenarios prone to such issues, which is less of a concern in Linear Regression. Figure 2.11 illustrates the trajectories of these three Gradient Descent techniques during training. While Batch GD precisely reaches the minimum, Stochastic GD and Mini-batch GD tend to oscillate nearby. It is important to note, however, that while Batch GD is slow in taking steps, both Stochastic GD and Mini-batch GD could also effectively reach the minimum with an appropriate learning rate schedule.

2.3 Feature Scaling

The cost function can resemble an elongated bowl if the feature scales vary significantly. Figure 2.17 illustrates the effect of feature scaling on Gradient Descent. In figure 2.17(a), Gradient Descent progresses directly towards the minimum, reaching it swiftly. However, in figure 2.17(b) it initially moves almost orthogonally to the direction of the global minimum, followed by a lengthy traverse down a nearly flat valley, significantly delaying the convergence.

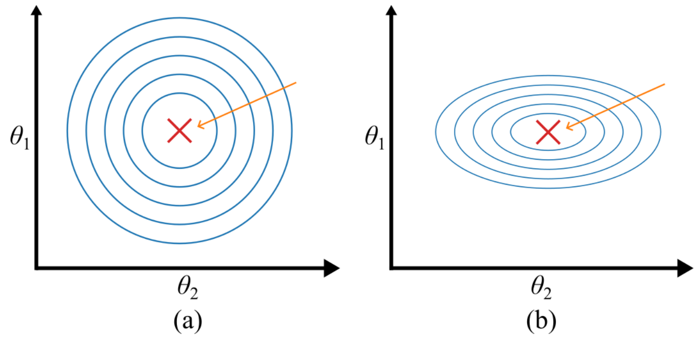

Figure 2.17: Gradient descent on a 2D surface for parameters that are: (a) equally scaled, and (b) unequally scaled.

Training a model can be viewed as a search through the parameter space for the combination that minimizes the cost function. When additional parameters are introduced, this space gains extra dimensions, making the optimization task more challenging. In Ordinary Least Squares Linear Regression, however, the cost surface is convex and bowl-shaped, so any descent method is guaranteed to reach the global minimum. To normalize feature scales, two common methods are employed. The are Min-max scaling and Standardization.

- **Min-max scaling**, often referred to as normalization, is straightforward: it rescales the data to a range of 0 to 1 by subtracting the minimum value and dividing by the range (max minus min).

$$\mathbf{X}' = \frac{\mathbf{X} - \mathbf{X}_{\text{min}}}{\mathbf{X}_{\text{max}} - \mathbf{X}_{\text{min}}} \tag{2.20}$$

  Scikit-Learn offers `sklearn.preprocessing.MinMaxScaler` for this purpose.

- **Standardization** differs significantly as it first subtracts the mean (resulting in a zero mean) and then divides by the standard deviation to achieve unit variance. This method does not limit values to a specific range, which can be problematic for certain algorithms, such as neural networks which often expect inputs between 0 and 1. However, standardization is less sensitive to outliers. For instance, if a median income is mistakenly recorded as 100, min-max scaling would compress all other values between 0 and 15 to between 0 and 0.15, whereas standardization would be minimally affected.

$$\mathbf{X}' = \frac{\mathbf{X} - \overline{\mathbf{X}}}{\sigma} \tag{2.21}$$

  `sklearn.preprocessing.StandardScaler` is provided by Scikit-Learn for standardization.

WARNING

It is crucial to apply scalers exclusively to the training data and not to the entire dataset, which includes the test set. This practice ensures that the model is not inadvertently exposed to test data during training. Once the scalers are fitted to the training data, they can then be used to transform the training set, the test set, and any new data subsequently encountered.

2.4 Polynomial Regression

What if the underlying pattern of your data is more complex than a simple linear relationship? Consider the dataset shown in figure 2.18 with non-linear characteristics (one feature across multiple instances), which typically can be modeled using a second-order polynomial

$$\hat{y} = ax^2 + bx + c. \tag{2.22}$$

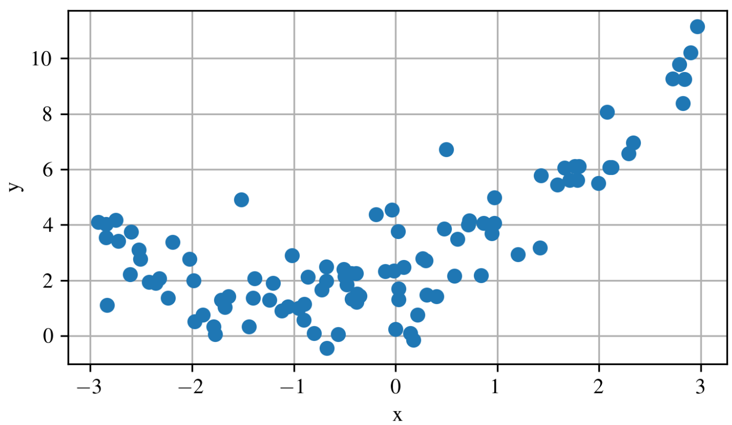

Figure 2.18: Dataset derived from a 2nd-order polynomial.

A simple linear fit will under-perform, so you first enrich the training set by adding the squared term of each feature as an extra column; shown in figure 2.19. Although the model you train is still linear with respect to its parameters, it now operates on an augmented feature space and can represent the desired quadratic curve. This approach is called Polynomial Regression, and it extends naturally by including higher powers whenever more complex nonlinearities are present.

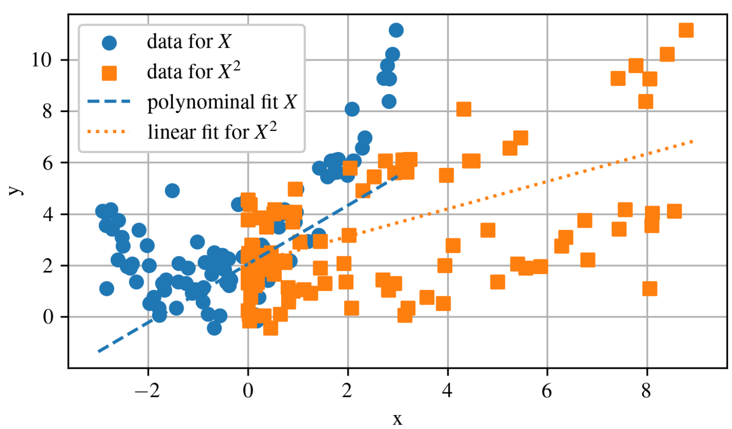

Figure 2.19: Linear models fit to the individual features of a polynomial dataset.

This data can then be incorporated into two linear models, where the slopes of the feature sets serve as the parameters for the base-line polynomial expression (a and b in Equation 2.22) and the bias term is the offset (c in Equation 2.22). The results of such a polynomial fit is shown in figure 2.20.

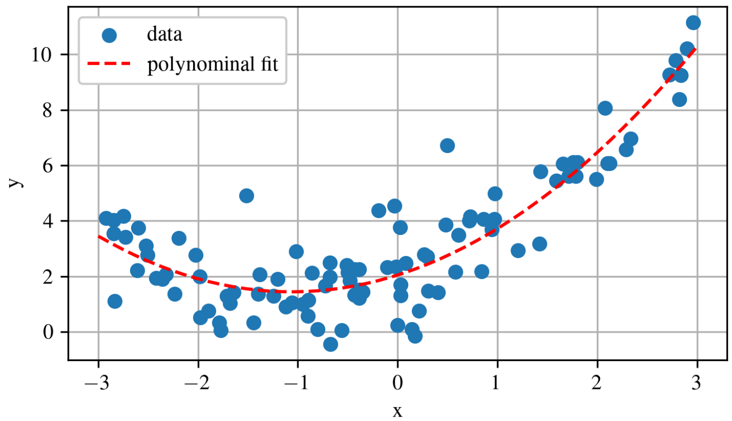

Figure 2.20: Polynomial model fit to a dataset.

It is important to note that Polynomial Regression can identify interrelationships between features in cases where multiple features exist, a capability beyond the scope of simple Linear Regression. This enhancement is enabled by `PolynomialFeatures`, which includes all possible combinations of features up to the specified degree. For instance, with two features a and b, and a degree of 3, `PolynomialFeatures` would add not only a^2, a^3, b^2, and b^3 but also the combined terms ab, a^2b, and ab^2.

WARNING

`PolynomialFeatures(degree=d)` expands an array with n original features into one that includes $\frac{(n+d)!}{d!n!}$ features, accounting for all combinations of features up to the d-th degree. Here, $n!$ represents the factorial of n, calculated as $1 \times 2 \times 3 \times \cdots \times n$. Be cautious of the rapid increase in the number of features, known as combinatorial explosion!

Example 2.2 Polynomial Regression

This example fits a non-linear dataset using polynomial regression by adding x^2 as an extra feature. It demonstrates how linear models can approximate curved relationships when polynomial terms are included, and shows the resulting fit compared to the original data.

2.5 Examples

Example 2.1

```python
"""
Example 2.1 Linear Regression
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import datasets
from sklearn.linear_model import LinearRegression

plt.close('all')

#%% load data

ames = sk.datasets.fetch_openml(name="house_prices", as_frame=True,parser='auto')
target = ames['target'].values
data = ames['data']
YrSold = data['YrSold'].values  # Year Sold (YYYY)
MoSold = data['MoSold'].values  # Month Sold (MM)
OverallCond = data['OverallCond'].values # OverallCond: Rates the overall condition of the house
GrLivArea = data['GrLivArea'].values # Above grade (ground) living area square feet
BedroomAbvGr = data['BedroomAbvGr'].values # Bedrooms above grade (does NOT include basement bedrooms)

# Ask a home buyer to describe their dream house, and they probably won't begin
# with the height of the basement ceiling or the proximity to an east-west railroad.
# But this playground competition's dataset proves that much more influences price
# negotiations than the number of bedrooms or a white-picket fence.

# With 79 explanatory variables describing (almost) every aspect of residential
# homes in Ames, Iowa, this competition challenges you to predict the final price
# of each home.

# Plot a few of the interesting features vs the target (price). In particular,
# let's plot the number of rooms vs. the price.
plt.figure()
plt.plot(GrLivArea,target,'o',markersize=2)
plt.xlabel('Above grade (ground) living area square feet')
plt.ylabel('price (USD)')
plt.grid(True)
plt.tight_layout()

#%% Build a model for the data
X = GrLivArea
Y = target
model_X = np.linspace(0,5000)

theta_1 = 0
theta_2 = 100
model_Y_manual = theta_1 + theta_2*model_X

plt.figure()
plt.plot(X,Y,'o',markersize=2,label='data')
plt.plot(model_X,model_Y_manual,'--',label='manual fit')
plt.xlabel('Above grade (ground) living area square feet')
plt.ylabel('price (USD)')
plt.grid(True)
#plt.xlim([3.5,9])
#plt.ylim([0,50000])
plt.legend(framealpha=1)
plt.tight_layout()

```

```
# add a dimension to the data as math is easier in 2d arrays and sk learn only
# takes 2d arrays
X = np.expand_dims(X,axis=1)
Y = np.expand_dims(Y,axis=1)
model_X = np.expand_dims(model_X,axis=1)

#%% compute the linear regression solution using the closed form solution

# compute
X_b = np.ones((X.shape[0],2))
X_b[:,1] = X.T # add x0 = 1 to each instance

theta_closed_form = np.linalg.inv(X_b.T@X_b)@X_b.T@Y

model_y_closed_form = theta_closed_form[0] + theta_closed_form[1]*3000
model_Y_closed_form = theta_closed_form[0] + theta_closed_form[1]*model_X

plt.figure()
plt.plot(X,Y,'o',markersize=3,label='data points')
plt.xlabel('Above grade (ground) living area square feet')
plt.ylabel('price (USD)')
plt.plot(3000,model_y_closed_form,'dr',markersize=10,zorder=10,
         label='inferred data point')
plt.plot(model_X,model_Y_closed_form,'-',label='normal equation')
plt.grid(True)
plt.legend()
plt.tight_layout()

#%% compute the linear regression solution using gradient descent

eta = 0.00000001 # learning rate
n_iterations = 100
m = X.shape[0]
theta_gradient_descent = np.random.randn(2,1) # random initialization
for iteration in range(n_iterations):
    gradients = 2/m * X_b.T.dot(X_b.dot(theta_gradient_descent) - Y)
    theta_gradient_descent = theta_gradient_descent - eta * gradients

print(theta_gradient_descent)

model_Y_gradient_descent =  theta_gradient_descent[0] \
    + theta_gradient_descent[1]*model_X

plt.figure()
plt.plot(X,Y,'o',markersize=3,label='data points')
plt.xlabel('Above grade (ground) living area square feet')
plt.ylabel('price (USD)')
plt.plot(model_X,model_Y_closed_form,'-',label='normal equation')
plt.plot(model_X,model_Y_gradient_descent,':',label='gradient descent')
plt.grid(True)
plt.legend()
plt.tight_layout()

#%% compute the linear regression solution using sk-learn

# build and train a closed from linear regression model in sk-learn
model_LR = sk.linear_model.LinearRegression()
model_LR.fit(X,Y[:,0])
model_Y_sk_LR = model_LR.predict(model_X)

# build and train a Stochastic Gradient Descent linear regression model in sk-learn.
# Note that in running the model, the best way to do this would be to use a pipeline
# =with feature scaling. However, here we just set 'eta0' to a low value, this
# is done only for educational # purposes and is not the ideal methodology in
# terms of system robustness.
model_SGD = sk.linear_model.SGDRegressor(learning_rate='constant',eta0=0.00000001)
model_SGD.fit(X,Y[:,0])
```

```
model_Y_sk_SGD = model_SGD.predict(model_X)

# plot the modeled results
plt.figure()
plt.plot(X,Y,'o',markersize=2,label='data')
plt.plot(model_X,model_Y_closed_form,'-',label='normal equation')
plt.plot(model_X,model_Y_gradient_descent,'--',label='gradient descent')
plt.plot(model_X,model_Y_sk_LR,':',label='sklearn normal equation')
plt.plot(model_X,model_Y_sk_SGD,'-.',label='sklearn stochastic gradient descent')

plt.xlabel('Above grade (ground) living area square feet')
plt.ylabel('price (USD)')
plt.grid(True)
plt.legend(framealpha=1)
plt.tight_layout()
```

Example 2.2

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Example 2.2
Polynomial regression
Machine Learning for Engineering Problem Solving
@author: Austin Downey
"""

import IPython as IP
IP.get_ipython().run_line_magic('reset', '-sf')

import numpy as np
import scipy as sp
import matplotlib as mpl
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn import linear_model

plt.close('all')

#%% build the data sets
np.random.seed(2) # 2 and 6 are pretty good
m = 100
X = 6 * np.random.rand(m,1) - 3
Y = 0.5 * X**2 + X + 2 + np.random.randn(m,1)

# plot the data
plt.figure()
plt.grid(True)
plt.scatter(X,Y)
plt.xlabel('x')
plt.ylabel('y')

#%% perform  polynomial regression

# generate x^2 as we use the model y = a*x^2* + b*x + c
X_poly_manual = np.hstack((X,X**2))

# or use the code as this does lots of features for multi-feature data sets.
poly_features = sk.preprocessing.PolynomialFeatures(degree=2, include_bias=False)
X_poly_sk = poly_features.fit_transform(X)

# they do do same thing as shown below, so select one to carry forward.
print(X_poly_manual == X_poly_sk)
X_poly = X_poly_manual

# In essence, we now have two data sets. We can plot that here
plt.figure()
plt.grid(True)
plt.scatter(X_poly[:,0],Y,label = 'data for x')
plt.scatter(X_poly[:,1],Y,marker='s',label = 'data for $x^2$')
plt.legend()
plt.xlabel('x')
plt.ylabel('y')

# and fit linear models to these data sets
model = sk.linear_model.LinearRegression() # Select a linear model
model.fit(X_poly,Y) # Train the model
X_model_1 = np.linspace(-3,3)
X_model_2 = np.linspace(0,9)

# the model parameters are:
model_coefficients = model.coef_
model_intercept = model.intercept_
print(model_coefficients)
print(model_intercept)
```

```python

Y_X1 = model_coefficients[0][0]*X_model_1 + model_intercept
Y_X2 = model_coefficients[0][1]*X_model_2 + model_intercept

# now if we plot the linear models on the extended set of features.
plt.figure()
plt.grid(True)
plt.scatter(X_poly[:,0],Y,label = 'data for x')
plt.scatter(X_poly[:,1],Y,marker='s',label = 'data for $x^2$')
plt.plot(X_model_1,Y_X1,'--',label='inear fit $x$')
plt.plot(X_model_2,Y_X2,':',label='linear fit for $x^2$',)
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.savefig('example_6_fig_1',dpi=300)

# now that we have a parameter for x and x^2, these can be recombined into a single
# model, y = x^2*a + x*b + c.
Y_polynominal = model_coefficients[0][1]*X_model_1**2 + model_coefficients[0][0]*\
    X_model_1 + model_intercept

plt.figure()
plt.grid(True)
plt.scatter(X,Y,label='data')
plt.plot(X_model_1,Y_polynominal,'r--',label='polynominal fit')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.savefig('example_6_fig_2',dpi=300)
```